

Lezione su RESTful API

Le **RESTful API** (Representational State Transfer) sono un tipo di architettura software utilizzata per progettare servizi web. REST è uno stile architettonico basato su principi ben definiti, che rende le API semplici, scalabili e facili da mantenere. In questa lezione, esploreremo i concetti fondamentali delle RESTful API, come funzionano e come progettarle.

1. Cos'è REST?

REST è uno stile architettonico per progettare sistemi distribuiti, in particolare applicazioni web. Si basa su alcuni principi chiave:

- **Risorse:** Tutto è una risorsa (ad esempio, un utente, un prodotto, un ordine).
 - **Identificatori univoci:** Ogni risorsa è identificata da un URL (Uniform Resource Locator).
 - **Metodi HTTP:** Le operazioni sulle risorse sono eseguite utilizzando i metodi HTTP (GET, POST, PUT, DELETE).
 - **Rappresentazioni:** Le risorse possono essere rappresentate in diversi formati (JSON, XML, ecc.).
 - **Stateless:** Ogni richiesta è indipendente e contiene tutte le informazioni necessarie per essere elaborata.
-

2. Metodi HTTP nelle RESTful API

I metodi HTTP definiscono le operazioni che possono essere eseguite sulle risorse:

Metodo HTTP	Descrizione	Esempio di utilizzo
GET	Recupera una risorsa o una lista di risorse.	Ottenere i dettagli di un utente.
POST	Crea una nuova risorsa.	Creare un nuovo utente.
PUT	Aggiorna una risorsa esistente (sostituzione).	Aggiornare i dettagli di un utente.
PATCH	Aggiorna parzialmente una risorsa.	Modificare solo l'email di un utente.

DELETE

Elimina una risorsa.

Eliminare un utente.

3. Endpoint RESTful

Un endpoint è un URL che rappresenta una risorsa o una collezione di risorse. Gli endpoint dovrebbero essere intuitivi e descrittivi.

Esempi di endpoint:

- GET /users → Ottieni tutti gli utenti.
 - GET /users/1 → Ottieni i dettagli dell'utente con ID 1.
 - POST /users → Crea un nuovo utente.
 - PUT /users/1 → Aggiorna l'utente con ID 1.
 - DELETE /users/1 → Elimina l'utente con ID 1.
-

4. Formati di rappresentazione

Le risorse possono essere rappresentate in diversi formati. I più comuni sono:

- **JSON** (JavaScript Object Notation): Leggero e facile da leggere.

```
{  
  "id": 1,  
  "nome": "Mario",  
  "cognome": "Rossi"  
}
```

- **XML** (eXtensible Markup Language): Più verboso, ma ancora utilizzato in alcuni contesti.

```
<user>  
  <id> </id>  
  <nome> </nome>  
  <cognome> </cognome>  
</user>
```

5. Stato e codici di risposta HTTP

Le RESTful API utilizzano codici di stato HTTP per indicare il risultato di una richiesta. Ecco alcuni esempi:

Codice	Descrizione
200	OK - Richiesta completata con successo.
201	Creato - Nuova risorsa creata con successo.
400	Bad Request - Richiesta malformata.
401	Unauthorized - Autenticazione richiesta.
404	Not Found - Risorsa non trovata.
500	Internal Server Error - Errore del server.

6. Esempio di API RESTful

Ecco un esempio di come potrebbe essere progettata una semplice API RESTful per gestire gli utenti:

Endpoint:

- GET /users → Ottieni tutti gli utenti.
- GET /users/{id} → Ottieni un utente specifico.
- POST /users → Crea un nuovo utente.
- PUT /users/{id} → Aggiorna un utente.
- DELETE /users/{id} → Elimina un utente.

Esempio di richiesta e risposta:

- **Richiesta GET /users/1:**

GET /users/1 HTTP/1.1

Host:

- **Risposta:**

HTTP/1.1 200 OK

Content-Type: application/json

{

```
"id": 1,  
"nome": "Mario",  
"cognome": "Rossi"  
}
```

- **Richiesta POST /users:**

POST /users HTTP/1.1

Host:

Content-Type:

```
{  
  "nome": "Luigi",  
  "cognome": "Verdi"  
}
```

- **Risposta:**

HTTP/1.1 201 Created

Content-Type: application/json

```
{  
  "id": 2,  
  "nome": "Luigi",  
  "cognome": "Verdi"  
}
```

7. Best Practices per RESTful API

- Usa nomi di risorse plurali (/users invece di /user).
 - Mantieni gli endpoint intuitivi e descrittivi.
 - Usa i codici di stato HTTP corretti.
 - Supporta la paginazione per le risorse di grandi dimensioni (es. /users?page=1&limit=10).
 - Utilizza JSON come formato principale per le risposte.
 - Proteggi le API con autenticazione (es. JWT, OAuth).
-

8. Strumenti per testare RESTful API

- **Postman:** Uno strumento popolare per testare e documentare le API.
 - **cURL:** Un comando da terminale per fare richieste HTTP.
 - **Swagger/OpenAPI:** Per documentare e testare le API in modo standardizzato.
-

9. Esempio di implementazione in Node.js

Ecco un esempio di una semplice API RESTful in Node.js utilizzando Express:

```
const express = require('express');
const app = express();
app.use(express.json());

let users = [
  { id: 1, nome: "Mario", cognome: "Rossi" },
  { id: 2, nome: "Luigi", cognome: "Verdi" }
];

// GET /users
app.get('/users', (req, res) => {
  res.json(users);
});

// GET /users/:id
app.get('/users/:id', (req, res) => {
  const user = users.find(u => u.id === parseInt(req.params.id));
  if (!user) return res.status(404).json({ message: "Utente non trovato" });
  res.json(user);
});

// POST /users
app.post('/users', (req, res) => {
  const newUser = { id: users.length + 1, ...req.body };
  users.push(newUser);
  res.status(201).json(newUser);
});

// PUT /users/:id
app.put('/users/:id', (req, res) => {
  const user = users.find(u => u.id === parseInt(req.params.id));
  if (!user) return res.status(404).json({ message: "Utente non trovato" });
  Object.assign(user, req.body);
  res.json(user);
});

// DELETE /users/:id
app.delete('/users/:id', (req, res) => {
  users = users.filter(u => u.id !== parseInt(req.params.id));
  res.status(204).send();
});
```

```
app.listen(3000, () => {  
  console.log('Server in ascolto sulla porta 3000');  
})
```

10. Conclusione

Le RESTful API sono uno standard diffuso per la creazione di servizi web scalabili e facili da usare. Seguendo i principi REST e utilizzando correttamente i metodi HTTP, è possibile progettare API robuste e intuitive. Con strumenti come Express (Node.js), Django REST Framework (Python) o Spring Boot (Java), puoi implementare API RESTful in modo efficiente.